Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Tarih: 8 Mart 2023 Çarşamba. Online ders (Zoom). Konu: MMDS kitabı 2. bölüm, MapReduce ve onu destekleyen yazılım yığını. Anlatım çoğunlukla İngilizce. Soruların cevapları ve ikinci yarının bir kısmı Türkçe. Ortada 10 dakikalık ara var.

Genel Konular

- Tek makinede büyük veri işlemenin sınırları: bellek ve disk okuma hızı.
- Küme hesaplama (cluster computing): raf kabinleri (rack), anahtarlar (switch), sıradan (commodity) Linux sunucular.
- Düğüm arızaları (node failure) ve uzun süren hesaplamalarda arıza.
- Ağ darboğazı (network bottleneck) ve dağıtık programlamanın karmaşıklığı.
- Dağıtık dosya sistemi (distributed file system, DFS): parça sunucuları (chunk server), ana düğüm (master node / NameNode), istemci kütüphanesi (client library).
- Hangi veri DFS'de tutulur: çok büyük, nadiren güncellenen, genelde sona eklenen (append) veri.
- Kelime sayma (word count) örneği: Linux komut zinciri ve MapReduce karşılığı.
- MapReduce: Map, Group by Key (shuffle/sort), Reduce.
- Diğer örnekler: alan adına (host) göre toplam boyut, 5 kelimelik dizileri sayma.
- MapReduce ortamının arka planda yaptıkları: bölümleme (partitioning) fonksiyonu, zamanlama, arıza yönetimi, makineler arası iletişim.
- Ara sonuçların (intermediate) yerel diskte tutulması.
- Ana düğümün koordinasyonu ve Map / Reduce / master arızalarında yapılanlar.
- Map ve Reduce görevi sayısının seçimi.

Hocanın Özellikle Vurguladığı Kısımlar

- CPU yalnız fiziksel bellekteki veriyle çalışır. Disk ile CPU arasında doğrudan bağlantı yoktur. Bu yüzden belleğe sığmayan veri birden fazla makineye bölünmelidir.
- Hesaplamayı veriye taşı, veriyi hesaplamaya taşıma. Kodun boyutu verinin boyutundan çok küçüktür. Ağ üzerinden yalnız ara sonuçlar taşınır.
- MapReduce üç sorunu çözer:
  - Veriyi birden fazla düğümde yedekli (redundant) saklayarak kalıcılık (persistency) ve erişilebilirlik sağlar.
  - Hesaplamayı verinin yanına taşıyarak veri hareketini en aza indirir.
  - Basit bir programlama modeliyle dağıtık programlamanın karmaşıklığını gizler.
- Programcı yalnız iki fonksiyon yazar: `map` ve `reduce`. Gruplama (group by key), taşıma, zamanlama ve arıza yönetimi kütüphaneye (Hadoop, Spark vb.) aittir.
- Group by key adımı hiçbir hesaplama yapmaz. Yalnız aynı anahtarlı (key) çiftleri bir araya getirir.
- Ara anahtar-değer çiftleri (intermediate key-value pairs) DFS'ye değil, Map'in çalıştığı makinenin yerel diskine yazılır. DFS'ye yazılsaydı gereksiz yere 2-3 kopya tutulur, hem yer hem zaman kaybedilirdi.
- Girdi ve son çıktı DFS'de tutulur. Böylece Reduce sonucu da kopyalanmış olur.
- Tamamlanmış bir Reduce görevi, sunucusu bozulsa bile tekrar çalıştırılmaz. Çünkü çıktısı DFS'ye yazılmış ve kopyalanmıştır. Map görevi ise tamamlanmış olsa da tekrar çalıştırılır. Çünkü çıktısı bozulan makinenin yerel diskindedir.

- Map sayısı genelde düğüm sayısından fazladır (her veri parçası için bir Map). Reduce sayısı genelde Map sayısından azdır. İkisi de yapılandırılabilir.

Kısa Tekrar Notları

- 10 milyar web sayfası × 20 KB = 200 TB. 50 MB/s disk okuma hızıyla tek makinede okumak 46 günden fazla sürer.
- Bir raf kabininde 16-64 sıradan Linux düğümü bulunur. Kabin içi anahtar 1 Gbps, kabinler arası omurga (backbone) anahtar 10 Gbps hız sağlar.
- Bir sunucu yaklaşık 3 yıl (1000 gün) dayanır. 1000 sunuculu kümede günde yaklaşık 1 arıza, 1 milyon sunuculu kümede günde yaklaşık 1000 arıza beklenir.
- DFS'de dosya parçalara (chunk) bölünür. Parça boyutu genelde 16-64 MB'tır (HDFS'te tipik olarak 64 MB). Her parça 2 veya 3 kopya olarak tutulur. Kopya sayısı yapılandırılabilir.
- Kopyalar farklı raf kabinlerine konur. Böylece bir kabinin anahtarı bozulsa bile veriye erişilir.
- Ana düğüm (HDFS'te NameNode) dosya kimliklerini ve parçaların hangi sunucularda olduğunu (metadata) tutar. Ana düğümün de kopyası tutulur.
- `map`: (anahtar, değer) alır, sıfır veya daha fazla ara (anahtar, değer) çifti üretir.
- `reduce`: (ara anahtar, değer listesi) alır, sonuç üretir.
- Bölümleme fonksiyonu: Ara anahtarı bir hash fonksiyonuna sokar. Sonuç, çiftin hangi Reducer'a gideceğini belirler.
- Görev durumları: boşta (idle), sürüyor (in progress), tamamlandı (completed).

Detaylı Açıklamalar

Neden küme hesaplama?

- Masaüstü ve dizüstü bilgisayarlarda çalışan uygulamaların verisi fiziksel belleğe sığar. Sığmazsa diskten belleğe getirilir.
- Büyük veri tek makinenin belleğine ve diskine sığmaz. Sığsa bile disk okuma hızı darboğaz olur (200 TB örneği).
- Bu derste klasik veri madenciliği ve makine öğrenmesi yöntemlerinin birden fazla makinede çalışan biçimleri incelenir.
- Küme yapısı:
  - Her raf kabininde sıradan sunucular vardır. Kabin içindeki sunucular bir anahtarla bağlıdır.
  - Kabinler bir omurga anahtarla birbirine bağlanır.
  - Veri sunuculara dağıtılır ve erişilebilirlik için kopyalanır.

Küme hesaplamanın iki sorunu

- Düğüm arızaları:
  - Sunucular özel tasarlanmış değildir. Masaüstü bilgisayarla aynı anakart ve kartları kullanır.
  - Kopyası olmayan bir parça, sunucusu bozulursa kaybolur. Bu yüzden veri kalıcı ve erişilebilir biçimde saklanmalıdır.
  - Saatlerce süren bir hesaplama sırasında düğüm bozulabilir. Hesaplama başka bir düğümde yeniden başlatılmalıdır.
- Darboğazlar:
  - Ağ darboğazı: Aynı kabindeki iki düğüm arasında büyük veri taşımak çok uzun sürer (hoca "yaklaşık bir gün" diyor, altyazı bu noktada bozuk). Çözüm: hesaplamayı verinin olduğu yere taşımak.
  - Dağıtık programlama karmaşıktır. Ağ yapısı ve soket programlama bilmeden kod yazabilmek için bir programlama modeli gerekir.

Dağıtık dosya sistemi

- Örnekler: Hadoop Distributed File System (HDFS), Google'ın dosya sistemi (Google File System, GFS).

- DFS'nin sağladıkları:
  - Dosya ad alanı (namespace): Her dosyaya tekil bir kimlik verilir. Kullanıcı bu kimlikle dosyayı ister. Sistem parçaları bulur, birleştirir ve bütün dosyayı verir.
  - Yedeklilik (redundancy): Her parça birden fazla sunucuda tutulur.
  - Erişilebilirlik (availability): Bir sunucu bozulsa da dosyaya erişilir.
- DFS'de tutulan veri:
  - Çok büyük dosyalar (yüzlerce GB, TB, PB).
  - Nadiren yerinde güncellenen veri. Örnek: log dosyaları. Log, olmuş bir olayın kaydıdır (after-the-fact data). Geçmiş değiştirilmez.
  - Yeni veri mevcut verinin sonuna eklenir (append).
  - Ana işlem okumadır. Örnek: Son 6 ayın log dosyalarında sayma işlemi.
- İlişkisel veritabanları (MSSQL, MySQL, PostgreSQL) ise sık güncellenen operasyonel veri içindir. Örnek: kayıt sistemi, sipariş sistemi, kullanıcının adres ve telefon bilgisi.
- Dropbox, Google Drive, OneDrive birer dağıtık depolamadır. Bir kişinin verisi küçük olsa da tüm kullanıcıların verisi toplamda çok büyüktür.

DFS bileşenleri

- Parça sunucuları (chunk server):
  - Dosya parçalarını tutar.
  - Örnek: C dosyası 6 parçaya (C0-C5), D dosyası 2 parçaya (D0, D1) bölünüp her parça 2 kopya olarak farklı sunuculara konmuş.
- Ana düğüm (master node, HDFS'te NameNode):
  - Dosya kimliklerini ve parçaların konumunu tutar.
  - Birden fazla kopyası tutulur.
- İstemci kütüphanesi (client library):
  - Uygulama, DFS'ye dosya yazmak veya okumak için bu kütüphaneyi kullanır.
  - Kütüphane, uygulamanın çalıştığı makinededir. Ana düğümle ve parça sunucularıyla doğrudan konuşur.

Örnek: Parçalar üzerinde log sayma (soru-cevap)

- Soru: Hesaplamayı verinin olduğu yerde yapıyorsak parçalar dağınıkken sonucu nasıl buluruz?
- Milyonlarca satırlık bir log dosyası 4 parçaya bölünmüş, her parçanın 3 kopyası var.
- Log kayıtlarını sayan kod 4 parçanın bulunduğu sunuculara gönderilir. Her sunucu kendi parçasındaki kayıtları sayar.
- Yalnız 4 ara sonuç tek bir sunucuya taşınır ve birleştirilir. Veri taşınmaz, sonuç taşınır.

Kelime sayma örneği

- Problem: Çok büyük bir metin dosyasında her farklı kelimenin kaç kez geçtiğini bulmak.
- Kullanım alanları:
  - Web sunucusu loglarında popüler URL'leri bulmak.
  - Arama motoru loglarında her sorgu ifadesinin kaç kez kullanıldığını bulmak.
- Sorun: Dosya belleğe sığmaz. Kelime-sayı çiftleri bile belleğe sığmayabilir. Bu yüzden bellekte bir hash tablosu tutmak yeterli değildir.
- Linux benzetmesi: `words(doc.txt) | sort | uniq -c`
  - `words`: Dosyadaki kelimeleri satır satır yazar (Map'e karşılık gelir).
  - `sort`: Kelimeleri sözlük sırasına göre sıralar (Group by key'e karşılık gelir).
  - `uniq -c`: Her kelimenin kaç kez geçtiğini sayar (Reduce'a karşılık gelir).
- MapReduce'un özü:
  - Map: Girdiyi kayıt kayıt tara, işe yarayan bilgiyi çıkar.
  - Group by key: Ara sonuçları sırala ve karıştır (shuffle).
  - Reduce: Sonuçları birleştir, özetle, çıktıyı yaz.

MapReduce adımları (kelime sayma)

- Girdi: (parça kimliği, parça içeriği) çiftleri. Değer, metin dosyasının bir parçasıdır.

- Map: Her parçadaki her kelime için (kelime, 1) çifti üretir. Örnek: (apple, 1), (orange, 1). Her parçada aynı Map fonksiyonu çalışır.
- Group by key: Kütüphane aynı anahtarlı çiftleri toplar. Örnek: (orange, [1, 1, 1]).
- Reduce: Her anahtarın değerlerini toplar. Örnek: (orange, 3).
- Sözde kod:

```
map(key, value):
    // key: belge adı; value: belge metni
    for each word w in value:
        emit(w, 1)

reduce(key, values):
    // key: kelime; values: sayılar
    result = 0
    for each count v in values:
        result += v
    emit(key, result)
```

- Map sayısı genelde parça sayısına eşittir. Reduce sayısı daha az olabilir. Örnek: 4 parça sunucusu, 4 Map, 3 Reduce.

Diğer örnekler

- Alan adına göre toplam boyut:
  - Girdi: URL, sayfa boyutu, tarih bilgisi içeren büyük bir derlem (corpus).
  - Map: Her URL'nin host adını bulur, (host adı, sayfa boyutu) üretir.
  - Reduce: Aynı host adının boyutlarını toplar.
- 5 kelimelik dizileri sayma:
  - Map: Belgedeki her ardışık 5 kelimelik diziyi çıkarır, (dizi, 1) üretir.
  - Reduce: Kelime saymadaki gibi toplar.

Soru-cevap: Map içinde saymak mümkün mü?

- Soru: Aynı kelimeleri Map aşamasında sayamaz mıyız?
- Cevap: Evet. Bu, birleştirici (combiner) fonksiyonudur ve bir sonraki derste işlenecek.
- Map tarafında yalnız o parçadaki yerel kelimeler sayılabilir. Örnek: Bir parçada 10 kelime, 4 farklı kelime varsa 10 çift yerine 4 çift gönderilir.
- Kazanç: Ara çiftleri Reducer'a taşırken harcanan ağ süresi azalır.

Soru-cevap: Reduce işini neden kütüphane yapmıyor?

- Reduce'ta yapılacak işlem probleme bağlıdır. Kelime saymada toplama, host örneğinde boyutları toplama yapılır. Bu yüzden Reduce'un içini programcı yazar.
- Kütüphane, verilen Map ve Reduce kodunu çalıştırır. Gruplama, sıralama ve ara çiftlerin Map sunucusundan Reduce sunucusuna taşınması kütüphanenin işidir.

Arka planda olanlar

- Her veri parçası için bir Map çalışır. Bir parça sunucusunda birden fazla Map olabilir.
- Map çıktıları yerel diske yazılır.
- Bölümleme fonksiyonu her ara anahtarı hash'ler. Sonuç, çiftin hangi Reduce görevine gideceğini belirler.
- Örnek kurulum: 5 sunucunun 3'ü Map, 2'si Reduce için kullanılmış. Reduce görevleri Map'lerin çalıştığı sunucularda da çalışabilir, ayrı sunucu zorunlu değildir.
- MapReduce ortamının görevleri:
  - Girdi verisini bölümlemek (parçalara bölme işini DFS yapar).
  - Programın makineler üzerinde çalışmasını zamanlamak: önce Map'ler, sonra taşıma, sonra Reduce'lar, en son çıktının DFS'ye yazılması.
  - Group by key adımını yürütmek.
  - Düğüm arızalarını yönetmek.
  - Makineler arası iletişimi yönetmek.

- Bir MapReduce görevinin çıktısı genelde başka bir MapReduce görevinin girdisi olur.

Ana düğümün koordinasyonu

- Her görevin durumunu tutar: idle, in progress, completed.
- Boşta olan görevler, işçi (worker) uygun oldukça zamanlanır.
- Map tamamlanınca ana düğüme ara dosyalarının konumunu, boyutunu ve hangi Reducer'a gideceğini bildirir.
- Ana düğüm bu bilgiyi ilgili Reducer'a iletir. Reducer dosyayı Map'in makinesinden kendi makinesine kopyalar, Reduce'u çalıştırır, sonucu DFS'ye yazar.
- Ana düğüm işçilere düzenli aralıklarla ping atarak hayatta olup olmadıklarını kontrol eder.

Arızalarla başa çıkma

- Map işçisi arızası: Tamamlanmış veya süren Map görevleri idle yapılır. Görev, aynı veri parçasının kopyasının bulunduğu başka bir düğümde yeniden çalıştırılır.
- Reduce işçisi arızası: Yalnız süren görevler idle yapılıp yeniden çalıştırılır. Tamamlanmış Reduce'un çıktısı zaten DFS'dedir.
- Ana düğüm arızası: Kopyası varsa işlem diğer ana düğümden devam eder. Tek ana düğüm varsa MapReduce görevi iptal edilir (abort) ve istemciye bildirilir.
- Soru-cevap: "Fonksiyonun fail olması" fonksiyonun çalıştığı sunucunun bozulması demektir.

Kendi HDFS kümeni kurmak

- Örnek: Masaüstü bilgisayar ve bir sunucu ile 2 düğümlü HDFS kümesi kurulabilir.
- HDFS'e her makinede ne kadar alan kullanacağı söylenir (örneğin 30 GB). Kalan alan makinenin yerel diskidir.
- MapReduce girdisi ve çıktısı HDFS'e konur. Ara çiftler yerel diskte tutulur ve iş bitince silinir.
- Map ve Reduce sayısı, parça sunucusu sayısı, ana düğüm kopyası gibi ayarlar yapılandırılabilir.

Ders sonrası notlar

- Hoca disk okuma hızını "50 megabit per second" dedi. Hesap 50 MB/s (megabyte) ile tutar: $200\ \text{TB}/50\ \text{MB/s} = 4 \times 10^6\ \text{s} \approx 46$ gün.
- `words` gerçek bir Linux komutu değildir. Kitaptaki örnekte dosyadan kelimeleri satır satır çıkaran varsayımsal bir betiktir. `sort` ve `uniq -c` gerçek komutlardır.
- Hoca ana düğümü ara ara "name node", "name server", "master" diye adlandırdı. Hepsi aynı koordinatör düğümü kastediyor. Hadoop'ta ise büyük olasılıkla NameNode yalnız HDFS üst verisini tutar; MapReduce görevlerini ayrı bir koordinatör (eski sürümlerde JobTracker, yenilerinde YARN ResourceManager) yönetir.
- Altyazının bozuk olduğu yerler (00:10:00, 00:17:30 civarı döngü, 00:48:00 civarı tek kelimelik bloklar, 01:05:30, 01:32:10 civarı) özetlenmedi.
- Gelecek hafta: MapReduce iyileştirmeleri (refinements), birleştirici (combiner) fonksiyonu.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.